

Telecommunications Engineers Training Cycle

Option

Cybersecurity and Defense

End of Study Project Report

Topic: Design and implementation of a Sybil attack detection algorithm in ITS

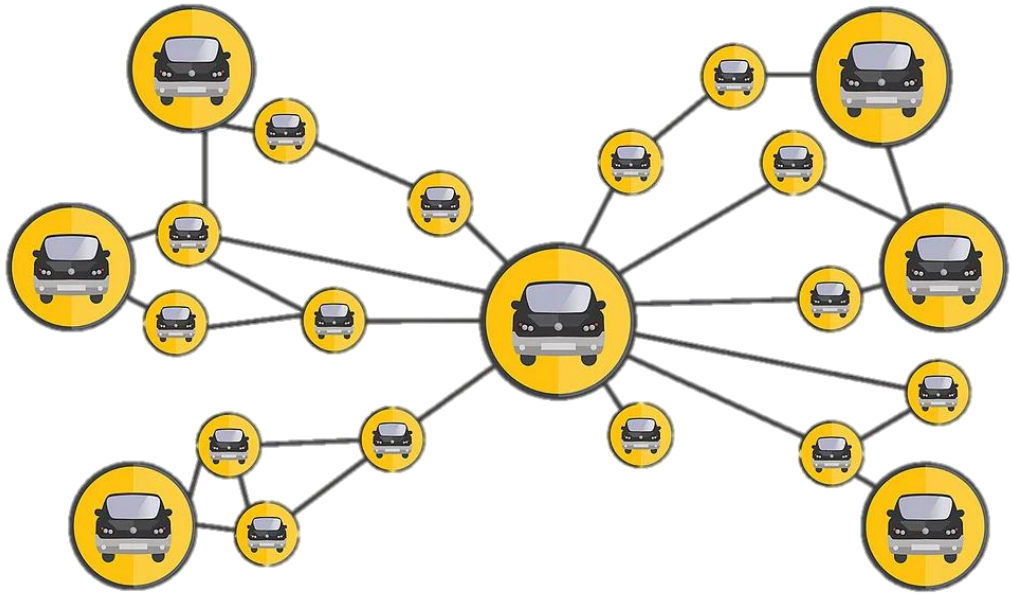
AUTHOR:
Dhia Eddine Laouiti

SUPERVISORS:
Mr. Marwane Ayaida & Mr. Messai Nadhir
Mme Najeh Sameh & Mme Najjar Leila

ACADEMIC YEAR:
2020-2021

Plan

1. Introduction
2. Proposed method
3. Implementation Results
4. VANET simulator
5. Conclusion & perspectives



Introduction



Project Host

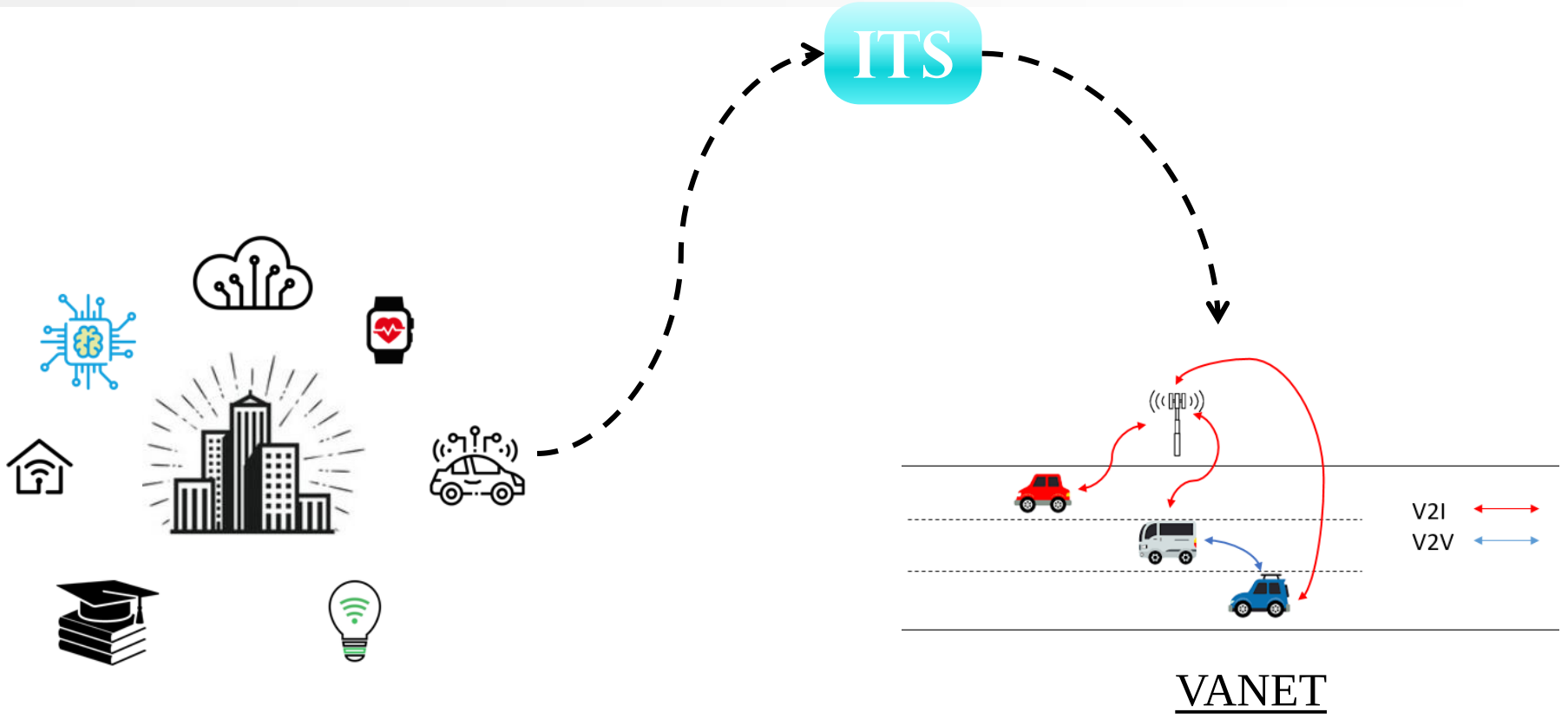


- University of Reims Champagne-Ardenne
- Imaging science research and innovation activities.

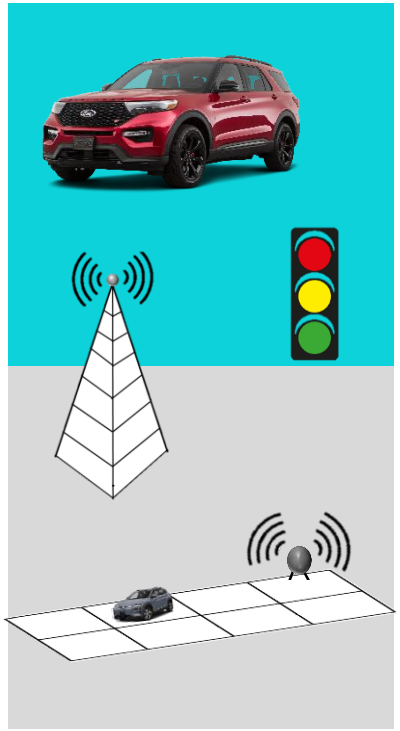


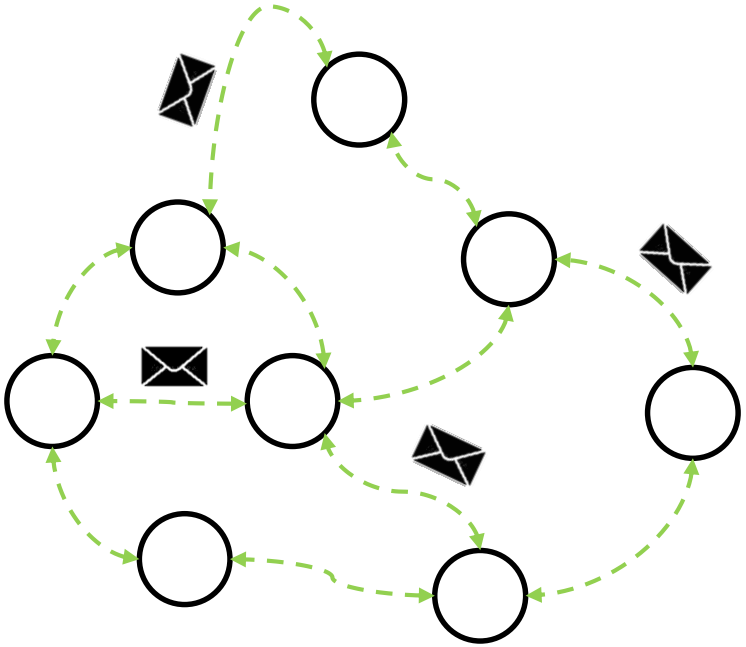
- SUP'COM
- Signal and image processing and numerical communication

Project Context



What is VANET?

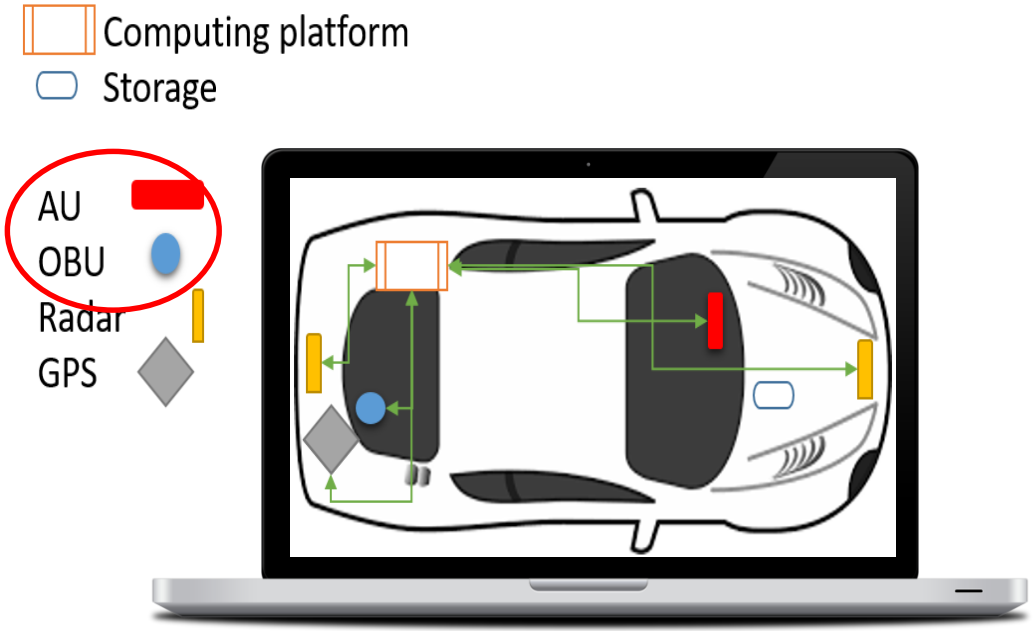


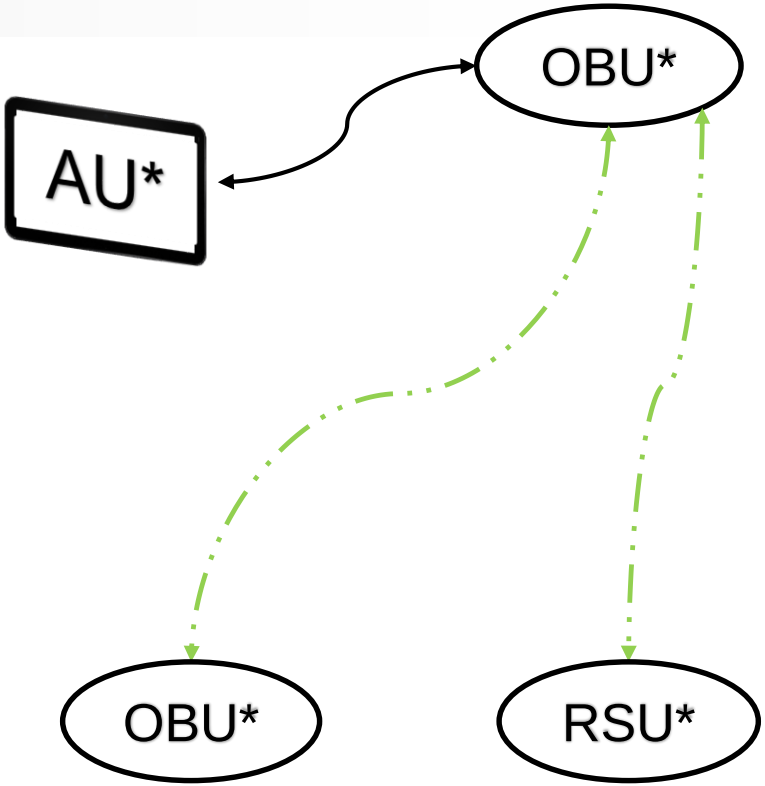


Intercommunication

- Variable mobility.
- Traffic density.
- Sufficient resources
- Environment.
- Security.

VANET architecture

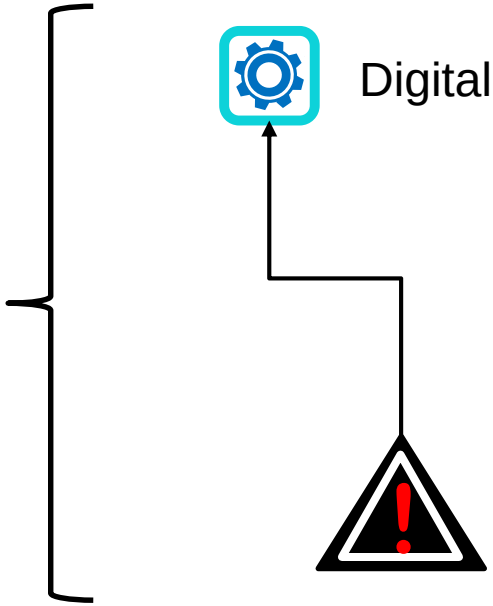




*AU: Application Unit
*OBU: On Board Unit
*RSU: Road Side Unit

Project presentation

Security problems

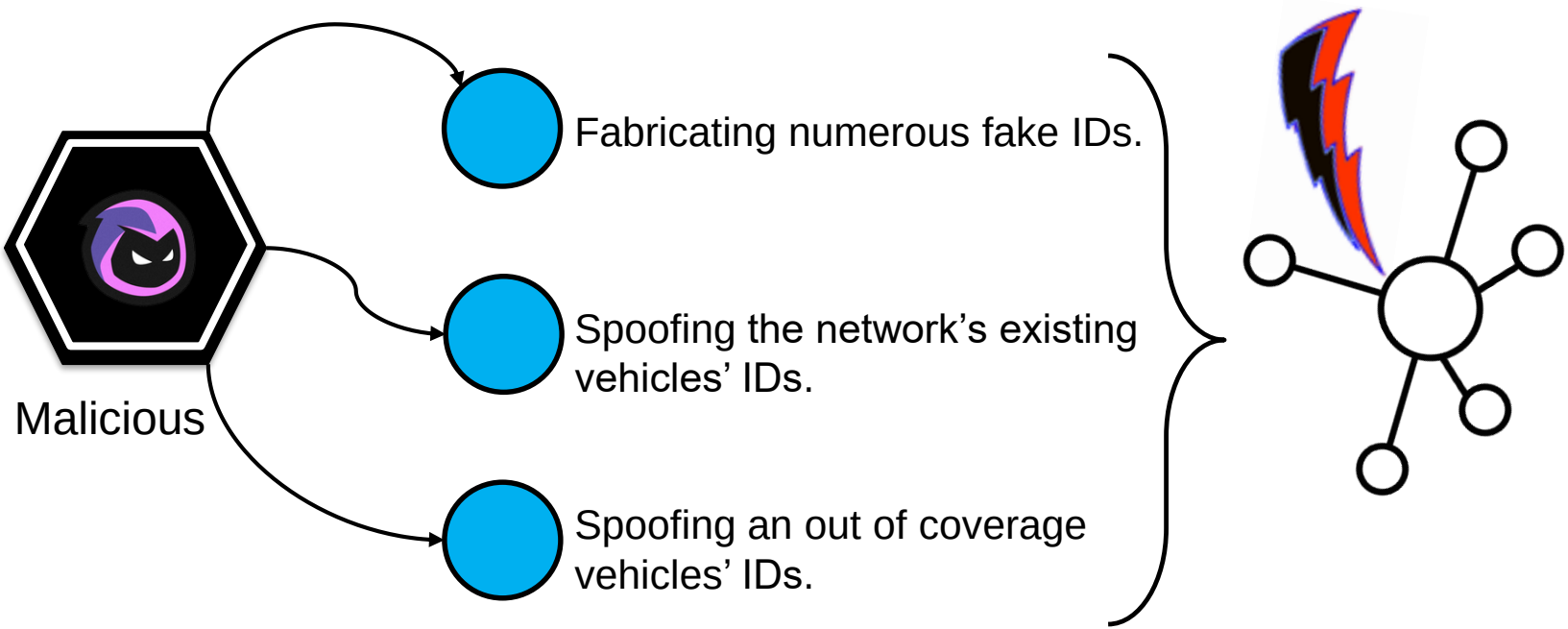


Digital identity

Threats and vulnerabilities

- Dos
- Black hole attack
- Malware
- Message delete
- Privacy breached
- Impersonation
- Message reply
- Sybil Attack

What is Sybil attack?

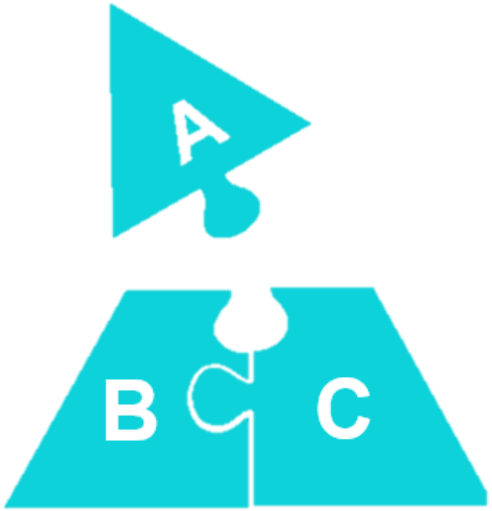


Solution ✓

A Machine learning

B Developping an algorithm

C Detecting sybil nodes



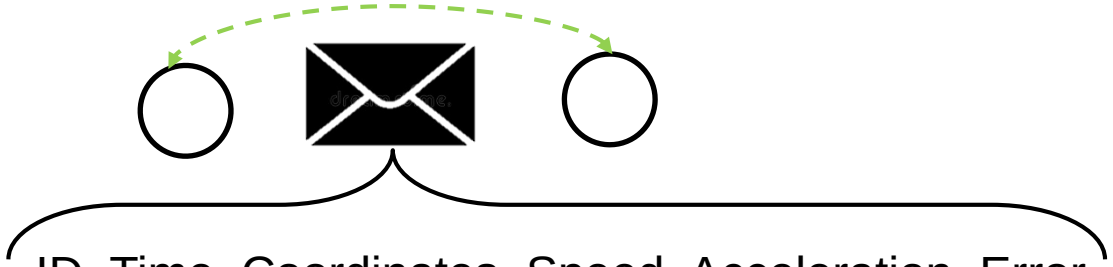
Proposed method



Applied preprocessing: DPM

Driving Pattern Matrix

$$M_i = \begin{vmatrix} m_{1,1} & m_{1,2} & m_{1,3} & m_{1,4} & m_{1,5} \\ m_{2,1} & m_{2,2} & m_{2,3} & m_{2,4} & m_{2,5} \\ m_{3,1} & m_{3,2} & m_{3,3} & m_{3,4} & m_{3,5} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ m_{n,1} & m_{n,2} & m_{n,3} & m_{n,4} & m_{n,5} \end{vmatrix}$$



ID, Time, Coordinates, Speed, Acceleration, Error



Creation of a Vector in t Time

Applied preprocessing: Eigenvalues

{ A * x = lambda * x }

$$\begin{bmatrix} LAMBDA2 & 0 & 0 \\ 0 & LAMBDA1 & 0 \\ 0 & 0 & LAMBDA \end{bmatrix}$$

$$M_i = \begin{vmatrix} m_{1,1} & m_{1,2} & m_{1,3} & m_{1,4} & m_{1,5} \\ m_{2,1} & m_{2,2} & m_{2,3} & m_{2,4} & m_{2,5} \\ m_{3,1} & m_{3,2} & m_{3,3} & m_{3,4} & m_{3,5} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ m_{n,1} & m_{n,2} & m_{n,3} & m_{n,4} & m_{n,5} \end{vmatrix}$$

$\{\lambda 1, \lambda 2\}$

Algorithm: computing Eigenvalues function

Require: Matrix of Movement M
Ensure: Two chosen eigenvalues
1. nligne ← number of rows in m
2. ncolon ← number of columns in m
3. for i ← 1 to ncolon do
4. $mean_i$ =sum of elements in the $column_i$ /nligne
5. end for
6. al ← 0
7. ac ← 0
8. while al < nligne do
9. while ac < ncolon do
10. $m_{al,ac} \leftarrow m_{al,ac} - mean_{ac}$
11. al ← al+1
12. ac ← ac+1
13. end while
14. end while
15. Msquare ← transposeM*M
16. eigenvalueslist ← eigenvalues(Msquare)
17. for j in eigenvalueslist do
18. $eigenvalueslist_j$ ← absolute value of $eigenvalueslist_j$
19. end for
20. sort in reverse eigenvalueslist
21. return first two values of eigenvalueslist

Database modification

	type	sendTime	sender	senderPseudo	messageID	pos	pos_noise	spd	spd_noise	
0	4	25202.602763	9	1092	4184	[265.1170585817953, 50.63668707563142, 0.0]	[3.044350425453742, 3.0129538268289062, 0.0]	[-0.075371354581445, 0.7293700027197371, 0.0]	[0.0, 0.0, 0.0]	[-0.2288 2.2145
1	4	25203.602763	9	1092	4194	[265.1699249881993, 52.59385797481798, 0.0]	[3.046216883546005, 3.125699247266999, 0.0]	[-0.298791556731674, 2.891411462036619, 0.0]	[-6.879720336200001e-05, 0.000665751818878, 0.0]	[-0.1860 1.80
2	4	25204.602763	9	1092	4204	[264.69109756979736, 56.50032206372517, 0.0]	[2.923726265209949, 3.17872844390584, 0.0]	[-0.523377030621331, 4.819057305062151, 0.0]	[-0.000272729919818, 0.0026392125160070003, 0.0]	[-0.18 1.7411
3	4	25205.602763	9	1092	4214	[263.6778228252816, 62.37854397335703, 0.0]	[2.993062400056588, 3.17906486636353, 0.0]	[-0.8834625063121231, 6.760168619201627, 0.0]	[-0.000477726268966, 0.004398722396264001, 0.0]	[-0.2569 1.966
4	4	25206.602763	9	1092	4224	[265.78328194613005, 70.67037405404739,	[3.023957070502346, 2.934935243060883,	[-1.3592378709377742, 8.787943591732375,	[-0.0008064038393340001, 0.0061705232424960005	[-0.2427 1.56

Rows: 368525 , columns: 13

	type	sendTime	sender	senderPseudo	messageID	pos	pos_noise	spd	spd_noise	
0	4	28800.472032	11067	10110672	42997460	[1287.7267225588794, 979.5779737417492, 0.0]	[4.747633520587553, 4.8645441143525865, 0.0]	[-1.748127667249639, -1.136239715434704, 0.0]	[-0.0, -0.0, 0.0]	[-1.535 -0.9979
1	4	28800.868894	11073	10110732	43001903	[293.58954769180605, 355.7927296040288, 0.0]	[4.032996657320159, 4.375615597609806, 0.0]	[1.915319325827097, 1.49976366675586, 0.0]	[-0.0, -0.0, 0.0]	[2.031 1.591
2	4	28801.368894	11073	10110732	43007697	[296.6292887821884, 358.09899031882077, 0.0]	[4.097822172584957, 4.153121720775444, 0.0]	[3.735322027289103, 2.91801740898278, 0.0]	[-0.002894874408843000 2, -0.002266790398687000...	[1.7258 1.3482
3	4	28801.472032	11067	10110672	43008699	[1284.6368951374668, 978.1973265103884, 0.0]	[4.850552123309093, 5.023051666806964, 0.0]	[-3.7717622720502932, -1.748085294360645, 0.0]	[0.004890223831438, 0.00317852445147, 0.0]	[-1.755 -0.813
4	4	28801.868894	11073	10110732	43011933	[296.6292887821884, 358.09899031882077	[4.097822172584957, 4.153121720775444	[3.735322027289103, 2.91801740898278	[-0.002894874408843000 2,	[1.7258 1.3482

Rows: 447617 , columns: 13

	senderPseudo	label	eignevalues_1	eignevalues_2
0	1	1	3.517500e+13	6.705461e+09
1	1092	0	5.872018e+10	1.581843e+07
2	10152	1	1.978002e+10	5.558777e+05
3	10212	0	7.412960e+10	1.050270e+07
4	10272	0	3.640613e+10	4.265857e+05
...	...	...	...	...
3472	60108752	1	5.140665e+10	1.419980e+06
3473	60109112	1	6.167463e+10	2.327020e+07
3474	60109712	1	3.335372e+10	5.000172e+06
3475	60110132	1	3.587088e+10	4.494635e+06
3476	60110312	1	6.678310e+09	1.039044e+04

Rows: 3476 , columns: 4

	senderPseudo	label	eignevalues_1	eignevalues_2
0	1	1	5.574952e+13	9.047051e+09
1	10110672	0	7.014396e+10	6.070092e+06
2	10110732	1	6.575150e+10	2.630144e+06
3	10110792	0	1.927563e+11	1.141317e+08
4	10110852	0	5.910607e+10	7.536634e+06
...	...	...	...	...
4294	60242313	1	4.193794e+10	1.144395e+06
4295	60242733	1	2.731844e+10	8.713054e+04
4296	60242913	1	9.472487e+09	2.627249e+05
4297	60243153	1	1.681605e+10	1.590477e+04
4298	60243333	1	1.579688e+10	3.513197e+05

Rows: 4298 , columns: 4

Implementation Results



Added features: Predicted location

Algorithm: computing predicted position function

Require: difference in time: T, current speed: S, current position: P, current acceleration: A
Ensure: the predicted position after the difference in time

1. predicted_position ← P+T*(S+T*A)
2. return predicted_position

predict_loc_x	predict_loc_y
1394.132581	1174.073460
1392.939850	1165.064122
123.814413	892.668626
913.335186	485.876813
915.519165	488.162305
...	...
1334.467598	1039.659189
1327.778757	1035.086219
1322.538126	1027.143545

Work methodology:



First Step : Implementing Machine learning algorithm



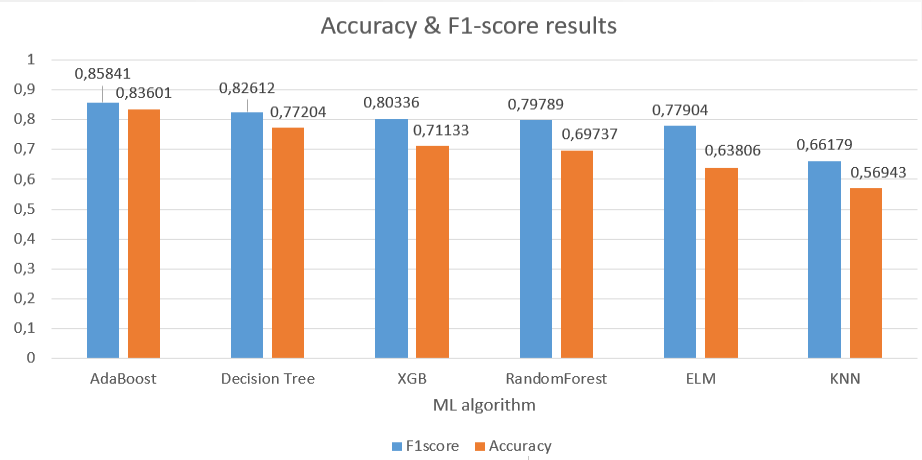
Second Step : Comparing between chosen matrices

- First list:['sendTime', 'pos_x', 'pos_y', 'predict_loc_x', 'predict_loc_y']
- Second list: ['sendTime', 'pos_x', 'pos_y', 'spd_x', 'spd_y', 'acl_x', 'acl_y', 'hed_x', 'hed_y', 'delta_acl_x', 'delta_acl_y']



Third Step : Proposed improvement

Implementing Machine learning algorithm



Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	94,6%	6,4%
Actual label 1	22%	78%

Confusion Matrix: Decision Tree	Predicted label 0	Predicted label 1
Actual label 0	64%	36%
Actual label 1	15%	85%

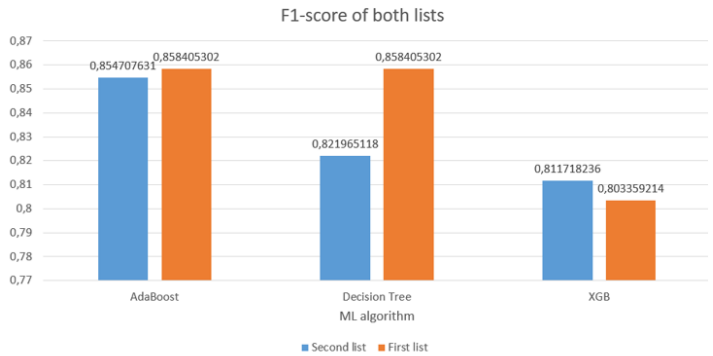
Confusion Matrix: XGB	Predicted label 0	Predicted label 1
Actual label 0	34%	66%
Actual label 1	7,6%	92,4%

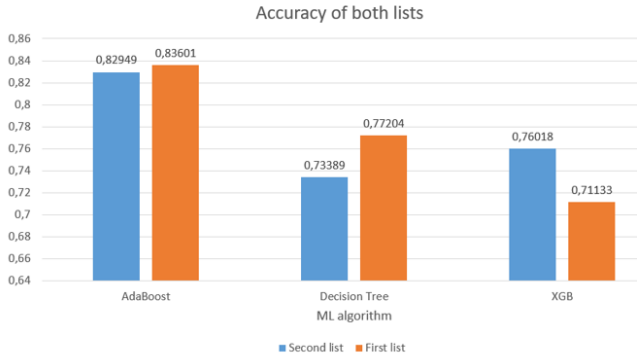
Confusion Matrix: KNN	Predicted label 0	Predicted label 1
Actual label 0	41%	59%
Actual label 1	34%	66%

Confusion Matrix: Random Forest	Predicted label 0	Predicted label 1
Actual label 0	28%	72%
Actual label 1	6,4%	94%

Confusion Matrix: ELM	Predicted label 0	Predicted label 1
Actual label 0	0%	100%
Actual label 1	0%	100%

Comparing between chosen matrices





First list:

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	94,6%	6,4%
Actual label 1	22%	78%

Confusion Matrix: Decision Tree	Predicted label 0	Predicted label 1
Actual label 0	64%	36%
Actual label 1	15%	85%

Confusion Matrix: XGB	Predicted label 0	Predicted label 1
Actual label 0	34%	66%
Actual label 1	7,6%	92,4%

Second list:

Confusion Matrix: AdaBoost	Predicted label 0	Predicted label 1
Actual label 0	91%	9%
Actual label 1	21%	79%

Confusion Matrix: Decision Tree	Predicted label 0	Predicted label 1
Actual label 0	57%%	43%
Actual label 1	13%	87%

Confusion Matrix: XGB	Predicted label 0	Predicted label 1
Actual label 0	44%	56%
Actual label 1	10%	90%

Proposed improvement: New feature

Algorithm: assign label depending on predicted position function

Require: Matrix of movement M
Ensure: list of labels on two axis (X , Y)
1. Sort the Matrix by ascending time
2. nli ← number of M rows
3. for j ← 1 to nli do
4. if difference between predicted position and current position on the X axis > 50 then
5. labelX ← 1
6. else labelX ← 0
7. end if
8. if difference between predicted position and current position on the Y axis > 50 then
9. labelY ← 1
10. else labelY ← 0
11. end if
12. add (labelX,labelY) to labellist
13. end for
14. return labellist

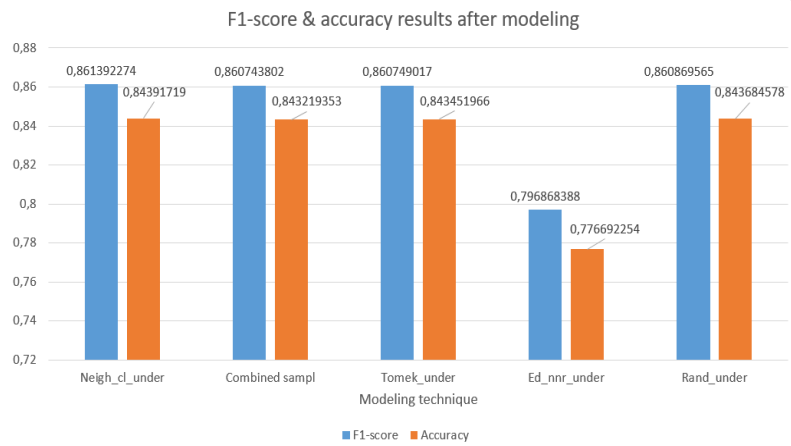
Algorithm: assign average of the labels from the predicted position function

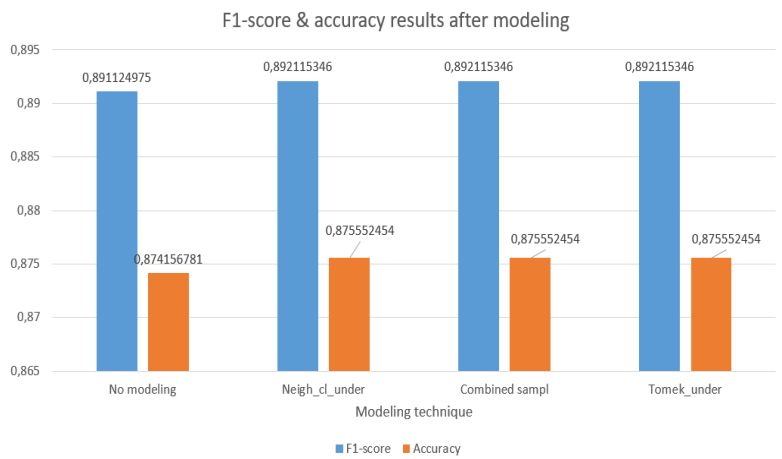
Require: dataset, list of the senderIDs in the dataset:Lsender
Ensure: list of labels of one element(0 or 1)
1. for sender in Lsender do:
2. select from dataset the rows of sender only
3. meanX ← calculate the average of prediction labels on X axis
4. meanY ← calculate the average of prediction labels on Y axis
5. meanlabel ← (meanX+meanY)/ 2
6. if meanlabel > 0 then
7. meanlabel ← 1
8. end if
9. add meanlabel to labellist
10. end for
11. return labellist

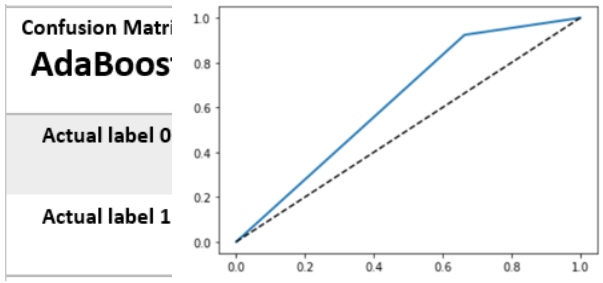
pred_label
1.0
0.0
0.0
0.0
0.0
...
1.0
1.0
1.0
1.0
0.0

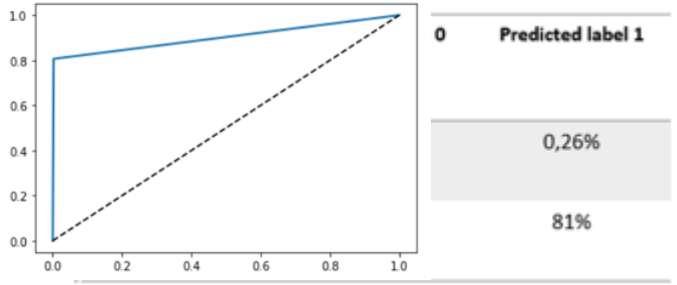
Modeling techniques

Using the plausibility label

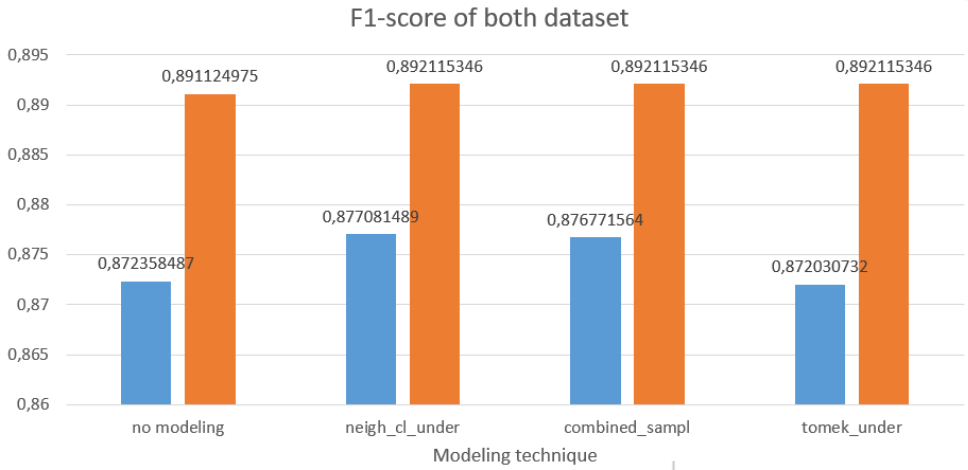


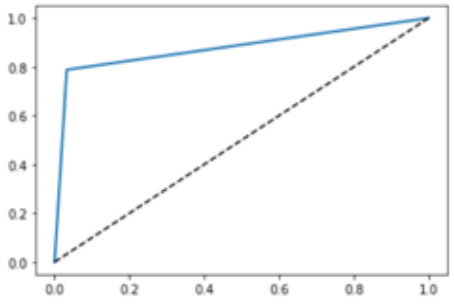




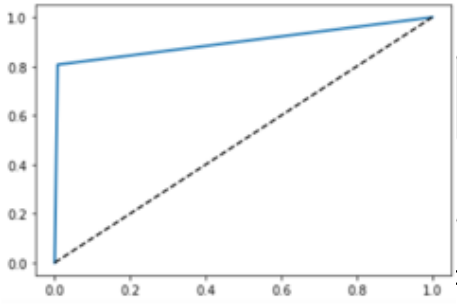


Final results of comparison





Brut Dataset



Created dataset from first list

Confusion Matrix:	Predicted label 0	Predicted label 1
	99,23%	0,77%
	19%	81%

Confusion Matrix:	Predicted label 0	Predicted label 1

:aset from first list

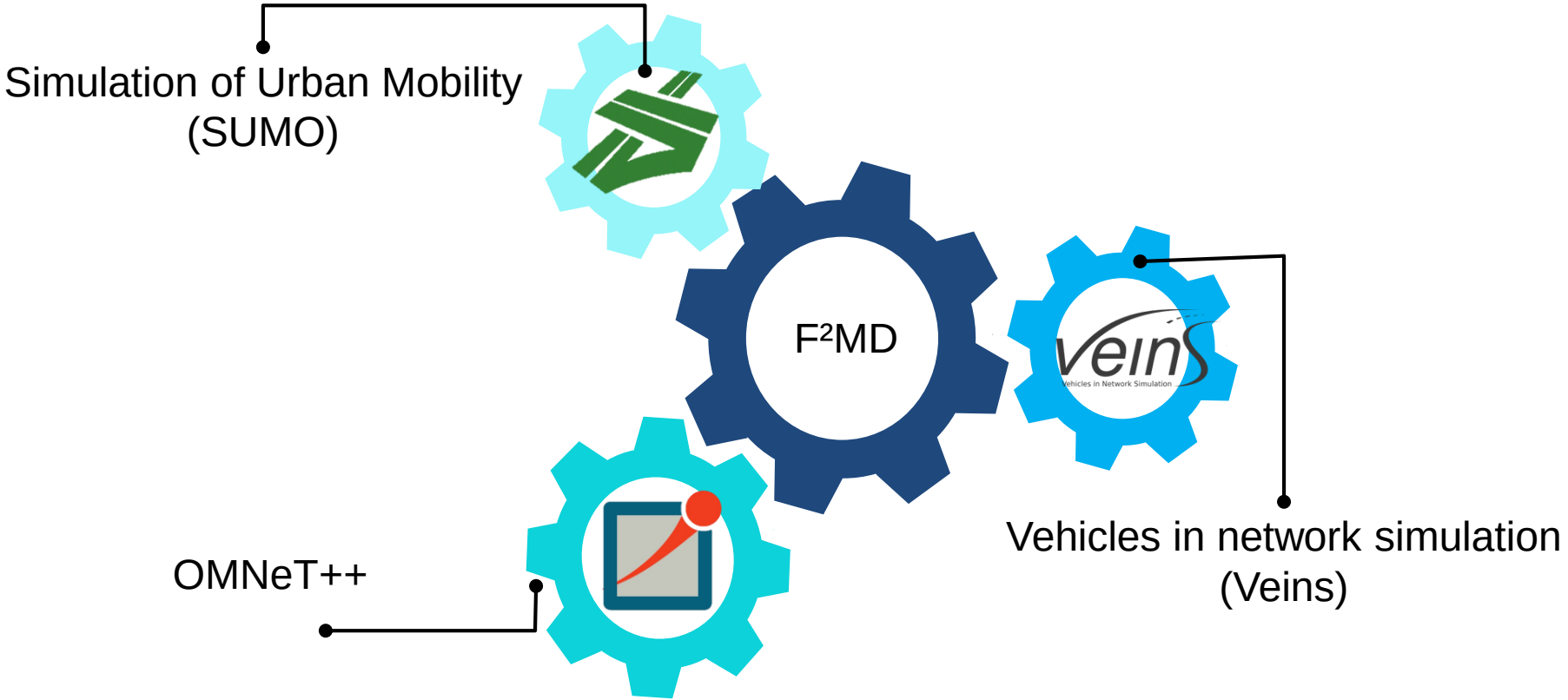
Project critic

Average of work time	Brut dataset	Refined dataset
AdaBoost (no modeling)	3min+20sec	0,92sec
AdaBoost (neigh_cl_under)	27min+29sec	0.94sec
AdaBoost (tomek_under)	26min+20sec	0.92sec
AdaBoost (combined_sampl)	3min+39sec	1sec

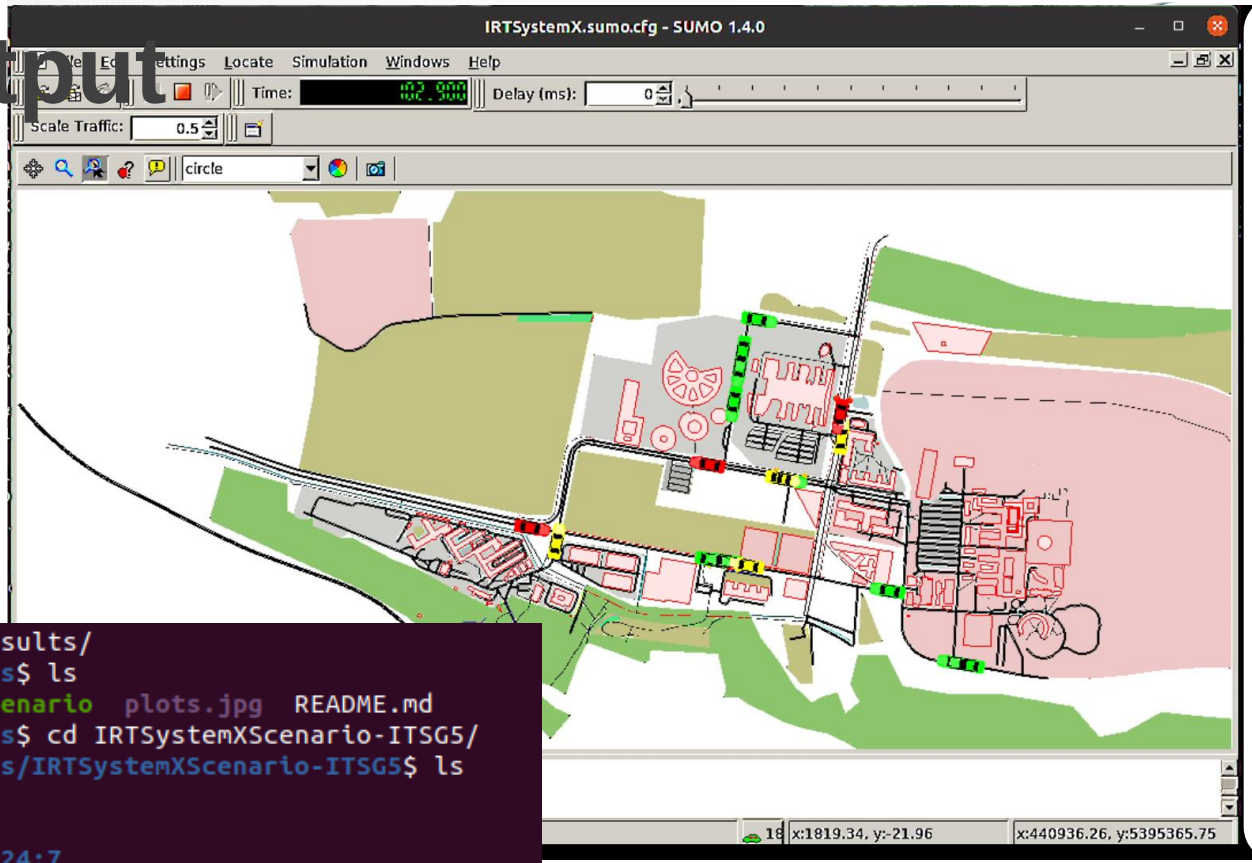
VANET simulator



Simulator environment



Simulation output




```
ubuntu@ubuntu-VirtualBox:~/F2MD$ cd f2md-results/
ubuntu@ubuntu-VirtualBox:~/F2MD/f2md-results$ ls
gui-plots  IRTSystemXScenario-ITSG5  plotScenario  plots.jpg  README.md
ubuntu@ubuntu-VirtualBox:~/F2MD/f2md-results$ cd IRTSystemXScenario-ITSG5/
ubuntu@ubuntu-VirtualBox:~/F2MD/f2md-results/IRTSystemXScenario-ITSG5$ ls
AppV1.dat           mdaV1.dat
AppV1Flags.dat      mdaV2.dat
AppV1FlagsInst.dat  MDBsms_V1_2021-9-21_12:24:7
AppV1Inst.dat       MDReports_V1_2021-9-21_12:24:7
AppV2.dat           VeReMi_0_3600_2021-9-21_12:37:2
AppV2Flags.dat      VeReMi_0_3600_2021-9-21_12:55:39
AppV2FlagsInst.dat  VeReMi_0_3600_2021-9-21_13:30:53
AppV2Inst.dat       VeReMi_0_3600_2021-9-23_5:5:1
ubuntu@ubuntu-VirtualBox:~/F2MD/f2md-results/IRTSystemXScenario-ITSG5$
```

F²MD detection process:

F²MD

Misbehavior authority server

Reports from simulation



Proposed modification

- Adding feature: vehicle reputation
- Adding algorithm:

Algorithm: predict the type of reported

Require: ID of the report sender, number of time the reportedID is reported, reputation of the senderID
Ensure: prediction if attacker or genuine

```
1.  N ← number of time the reportedID is reported
2.  If  reputation of the sender > 20
3.          predicted ← "Attacker"
4.  Else if reputation > -8 do
5.          if N > 5 do
6.                  predicted ← "Attacker"
7.          else
8.                  predicted ← "Genuine"
9.  Else
10.         N ← N-1
11.         if N > 5 do
12.                 predicted ← "Attacker"
13.         else
14.                 predicted ← "Genuine"
```

20 is threshold of a great reputation

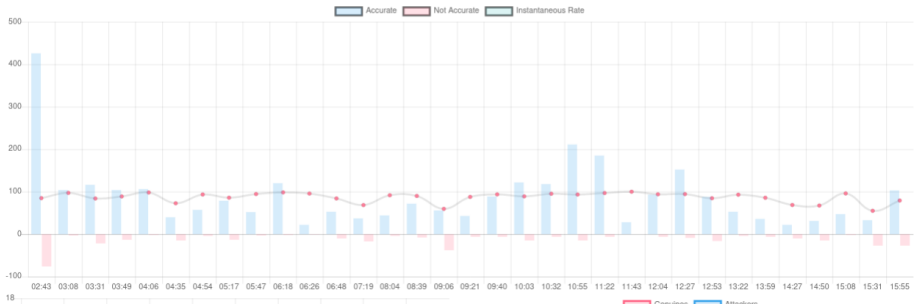
Range of threshold of normal reputation

5 is the threshold of reported times

Ignoring the generated report

Results

New Report Received!
Cumulative Reports Number: 3349
Cumulative Detection Rate: 87%



Confusion Matrix:	Predicted label 0	Predicted label 1
Actual label 0	79%	21%
Actual label 1	1%	99%

Precision: 0.99816

Recall: 0.70317

F1-score: 0.82509

Accuracy: 0.85955

Conclusion & perspectives



Conclusion



New acquired knowledge



- F1-score 89%
- Accuracy 87%



Research continuance

Thank you for your attention

